

## Experiment 06:

### 7. Laboratory Exercise

1. Attach the complete code performing key generation of both the parties.

#### Code:

```
def is_prime(number):
    """Return True if the given number is prime."""
    if number < 2:
        return False

    for divisor in range(2, int(number ** 0.5) + 1):
        if number % divisor == 0:
            return False

    return True

def calculate_public_key(generator, private_key, prime_number):
    """Calculate public key: Y = alpha^X mod q."""
    return pow(generator, private_key, prime_number)

def calculate_shared_key(public_key, private_key, prime_number):
    """Calculate shared secret: K = Y_other^X mod q."""
    return pow(public_key, private_key, prime_number)

|
print("----- Diffie-Hellman Key Exchange -----")

# Input global public parameters
prime_number = int(input("Enter prime number q: "))
generator = int(input("Enter primitive root alpha: "))

# Validate q
if not is_prime(prime_number):
    print("Error: q must be a prime number.")
    exit()

# Validate alpha
if generator <= 1 or generator >= prime_number:
    print("Error: alpha must satisfy 1 < alpha < q.")
    exit()

# Input private keys
alice_private_key = int(input("Enter Alice's private key XA: "))
bob_private_key = int(input("Enter Bob's private key XB: "))

# Validate private keys
if not (1 <= alice_private_key < prime_number):
    print("Error: Alice's private key XA must be smaller than q.")
    exit()

if not (1 <= bob_private_key < prime_number):
    print("Error: Bob's private key XB must be smaller than q.")
    exit()

# Calculate public keys
alice_public_key = calculate_public_key(
    generator,
```

```

    alice_private_key,
    prime_number
)

bob_public_key = calculate_public_key(
    generator,
    bob_private_key,
    prime_number
)

# Calculate shared secret keys
alice_shared_key = calculate_shared_key(
    bob_public_key,
    alice_private_key,
    prime_number
)

bob_shared_key = calculate_shared_key(
    alice_public_key,
    bob_private_key,
    prime_number
)

# Display results
print("\n----- Alice -----")
print("Private Key (XA):", alice_private_key)
print("Public Key (YA):", alice_public_key)

print("\n----- Bob -----")
print("Private Key (XB):", bob_private_key)
print("Public Key (YB):", bob_public_key)

print("\n----- Shared Secret -----")
print("Alice's Shared Key:", alice_shared_key)
print("Bob's Shared Key  :", bob_shared_key)

# Verify shared key
if alice_shared_key == bob_shared_key:
    print("\nShared key successfully established.")
else:
    print("\nError: Shared keys do not match.")

```

## 2. Attach the program output for key generation (display public key & private key for Both A & B) for the inputs given in all three exercises above.

a. Alice and Bob decide to use Diffie Hellman key exchange with  $q = 23$ ,  $\alpha = 7$ ,  $X_A = 3$  &  $X_B = 6$ . Find their public and private keys and the shared key.

```

----- Diffie-Hellman Key Exchange -----
Enter prime number q: 23
Enter primitive root alpha: 7
Enter Alice's private key XA: 3
Enter Bob's private key XB: 6

----- Alice -----
Private Key (XA): 3
Public Key (YA): 21

```

```

----- Bob -----
Private Key (XB): 6
Public Key (YB): 4

----- Shared Secret -----
Alice's Shared Key: 18
Bob's Shared Key  : 18

Shared key successfully established.

```

b. For  $q = 7, \alpha = 3, X_A = 2$  &  $X_B = 5$  Validate public keys, private keys and the shared key.

```
----- Diffie-Hellman Key Exchange -----
Enter prime number q: 7
Enter primitive root alpha: 3
Enter Alice's private key XA: 2
Enter Bob's private key XB: 5

----- Alice -----
Private Key (XA): 2
Public Key (YA): 2
```

```
----- Bob -----
Private Key (XB): 5
Public Key (YB): 5

----- Shared Secret -----
Alice's Shared Key: 4
Bob's Shared Key : 4

Shared key successfully established.
```

c. For  $q = 17, \alpha = 5, X_A = 4$  &  $X_B = 6$  Validate public keys, private keys and the shared key.

```
----- Diffie-Hellman Key Exchange -----
Enter prime number q: 17
Enter primitive root alpha: 5
Enter Alice's private key XA: 4
Enter Bob's private key XB: 6

----- Alice -----
Private Key (XA): 4
Public Key (YA): 13
```

```
----- Bob -----
Private Key (XB): 6
Public Key (YB): 2

----- Shared Secret -----
Alice's Shared Key: 16
Bob's Shared Key : 16

Shared key successfully established.
```

3. Attach the Screenshot taken for key generation using Cryptography virtual lab simulation.

### Step 1: Public Information Setup

Public information is shared openly between Alice and Bob. This includes a large prime number ( $p$ ) and a generator ( $g$ ). An eavesdropper can see these values, but they alone don't reveal the final shared secret.

Prime Number ( $p$ ):

Generate Prime Selects a large prime number for the key exchange

Generator ( $g$ ):

Another Generator Chooses a primitive root modulo  $p$

**Important:** Both Alice and Bob can see the same public values ( $p, g$ ). These are not secret and can be transmitted over an insecure channel.

**Step 2: Private Key Generation and Public Key Calculation**

Each participant generates their own private key (secret) and calculates their public key using the formula:  $\text{Public Key} = g^{\text{private key}} \bmod p$

**Alice**

Private Key (a):

Generate A

Alice's secret private key (kept secret)

Public Key ( $g^a \bmod p$ ):Calculate  $g^a$ 

Alice's public key (can be shared openly)

Send Public Key to Bob

Received from Bob:

Bob's public key ( $g^b \bmod p$ )

Shared Secret:

Calculate Shared Secret

Final shared secret:  $(g^b)^a \bmod p = g^{(ab)} \bmod p$ **Bob**

Private Key (b):

Generate B

Bob's secret private key (kept secret)

Public Key ( $g^b \bmod p$ ):Calculate  $g^b$ 

Bob's public key (can be shared openly)

Send Public Key to Alice

Received from Alice:

Alice's public key ( $g^a \bmod p$ )

Shared Secret:

Calculate Shared Secret

Final shared secret:  $(g^a)^b \bmod p = g^{(ab)} \bmod p$ **Step 3: Verification****Success: Alice and Bob derived the same shared secret.**

**Success!** If both Alice and Bob calculated the same shared secret value, they have successfully established a secure communication channel. This shared secret can now be used as an encryption key for further communication.

**Security Note:** Even though an eavesdropper can see all the public information ( $p, g, g^a \bmod p, g^b \bmod p$ ), they cannot easily compute the shared secret  $g^{(ab)} \bmod p$  without solving the discrete logarithm problem, which is computationally difficult for large numbers.